

Skeleton post processing software code document

Purpose

The purpose of the skeleton post processing software is to track the position and movement of the user. It takes a video input and interpret the users current activity based on their body. It tracks the user activity to help determine how much they are moving around and how long they are staying in one place. It can also track falling in the case of emergencies.

Libraries

- Opencv

Opencv is an open-source library for python which allows it to access a pictures or videos. It will be used to let the code see the user and process the video.

- Ultralytics YOLO

Ultralytics YOLO is an image recognition library used for image detection. YOLO processes the whole image which allows it to work quicker than other image detection.

- CSV

CSV library is used to save the activity data to the external csv file. This is important for collecting and storing the user's activity data.

- Time

This library allows for the program to measure time. This is used to calculate the velocity of the user for fall detection.

- Datetime

Datetime allows python to create dates and times in a more structured format. This is used when collecting the user activity data.

Video input

An important part of this software is its ability to track a person to be able to interpret their activities.

```
cap = cv2.VideoCapture(0) #access webcam. 0 tries for the webcam
```

The command VideoCapture is what is used to determine the input source for the program. It can be a video, or an image but if it set to 0, then it will default to the local camera, which during development was the webcam on my computer.

As the stream is a video it needs to be put into a while loop

```
26 while True:
27     ret, frame = cap.read()           #read the camera
28
29     if not ret:                       #if the camera is not accessed try again
30         cap = cv2.VideoCapture(0)
31         continue
32
```

The if statement here is a check to make sure the code can access the webcam. If it can't it will retry.

```
142 cv2.imshow('frame',frame)           #display the camera and the keypoints
```

This line creates a window which will display what the camera sees which looks like this:



Skeleton post processing

The next step is to have the YOLO model read the image for the keypoints on the person. The keypoints are important points on the body that are tracked by the model. They tend to be points of articulation like a shoulder or elbow for example.

```
24 while True:
25     ret, frame = cap.read()           #read the camera
26
27     if not ret:                       #if the camera is not accessed try again
28         cap = cv2.VideoCapture(0)
29         continue
30
31     #frame = cv2.resize(frame, (640,720)) #resizing code, uncomment to use
32
33     width, height = frame.shape[:2]    #copy width and height of the camera and create a blank space to place the keypoints on
34     blank = np.zeros((width, height, 3), dtype=np.uint8)
35
36     res = model(frame)                 #run the current frame through the model
37
38     for keypoints in res[0].keypoints.data: #plot each keypoint
39         keypoints = keypoints.cpu().numpy()
```

Inside the while loop, a blank square is also created along with getting the frame. This blank square is used to display the skeleton after it has been created. Afterwards the image is sent through the model, and an array of keypoints is created.

```
43 #DISPLAY KEYPOINTS
44 for i, keypoint in enumerate(keypoints):
45     x,y,confidence = keypoint
46     if confidence > 0.7:
47         cv2.circle(blank, (int(x), int(y)), radius=5,
48                     color=(255,0,0), thickness=1) #keypoint marked with circle
49
50         cv2.putText(blank, f'{i}', (int(x), int(y)-10),
51                     cv2.FONT_HERSHEY_SIMPLEX, 0.4, (255,255,255), 1,cv2.LINE_AA) #keypoints named with a number
52
```

This section plots the keypoints onto the blank square.

```
54
55 #LINES BETWEEN KEYPOINTS
56 connections = [           #lines between the numbered keypoints
57     (3,1), (1,0), (0,2),
58     (2,4), (1,2), (4,6),
59     (3,5), (5,6), (5,7),
60     (7,9), (6,8), (8,10),
61     (11,12), (11,13), (13,15),
62     (12,14), (14,16), (5,11),
63     (6,12)
64 ]
65
66 for part_a, part_b in connections: #plotting the connections
67     x1, y1, conf1 = keypoints[part_a]
68     x2, y2, conf2 = keypoints[part_b]
69
70     if conf1 > 0.5 and conf2 > 0.5: #drawing th lines onto the blank page
71         cv2.line(blank, (int(x1), int(y1)), (int(x2),int(y2)), (255,0,255),thickness=2)
72
```

This section of the code plots the lines between the keypoints. The array “connections” represents the lines between the keypoints. The code will then get the coordinates of the keypoints and plot the line.

```
143 cv2.imshow('Skeleton', blank)
```

This section creates a window to display the keypoints plotted on the blank square which looks like this:

